

QUEUES

Definition

- A **queue** is a linear list in which data can only be inserted at one end, called the **rear**, and deleted from the other end, called the **front**.
- Hence, the data are processed through the queue in the order in which they are received (**first in □ first out – FIFO**)

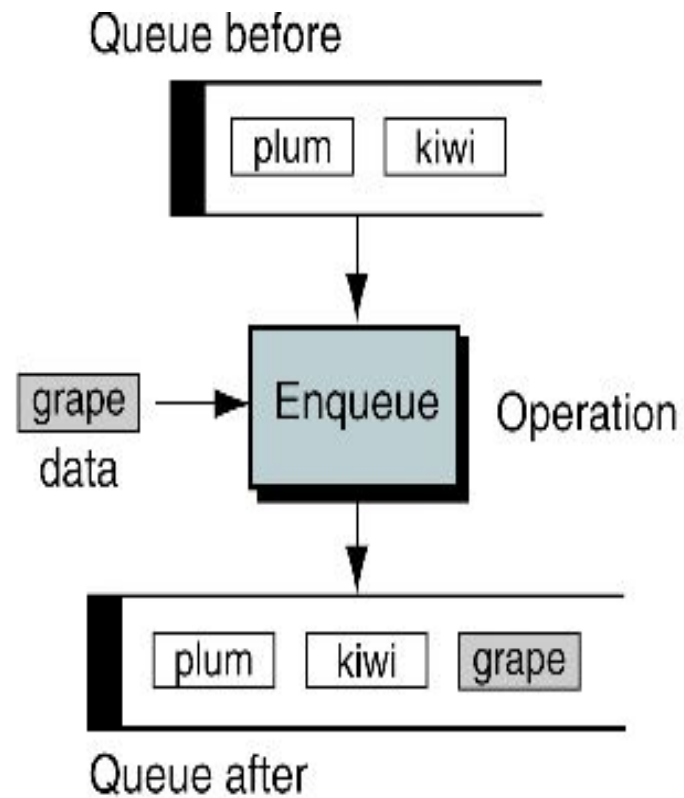


FIGURE 4-2 Enqueue

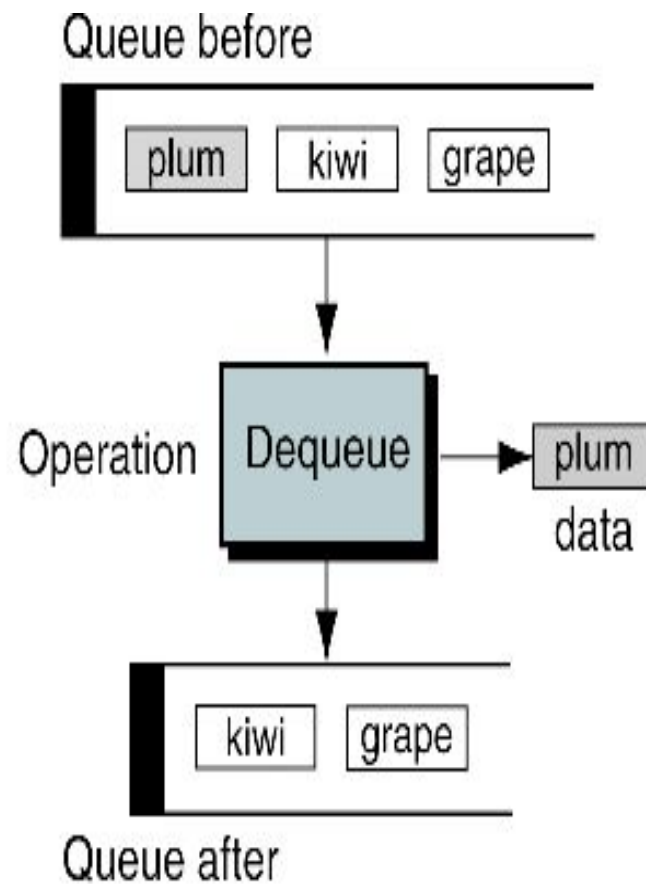


FIGURE 4-3 Dequeue

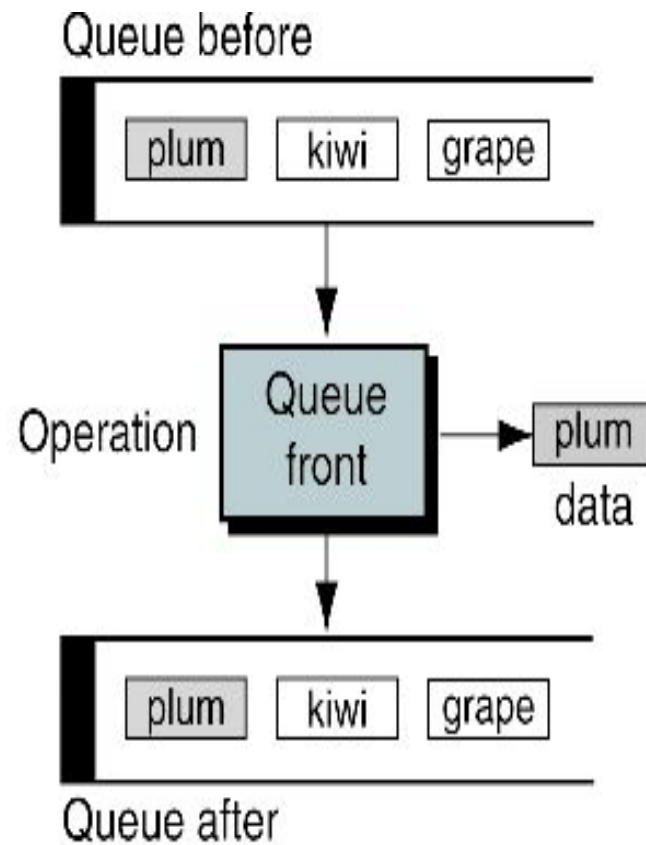


FIGURE 4-4 Queue Front

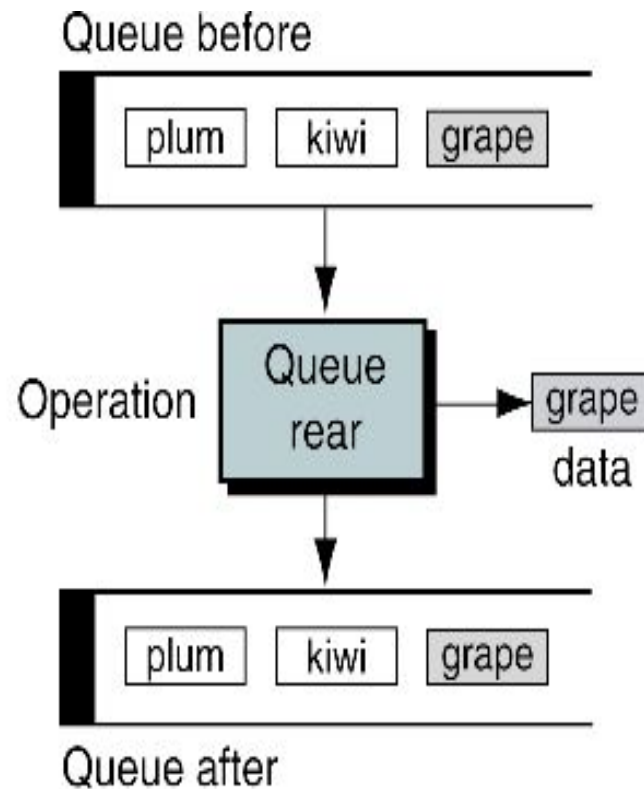
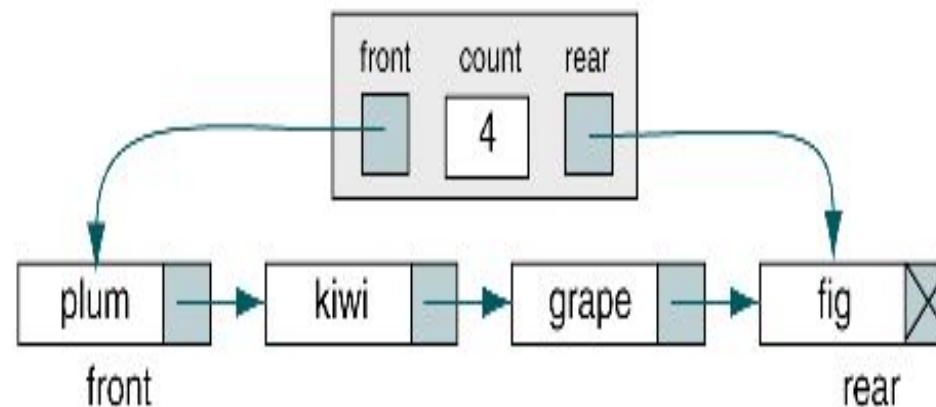


FIGURE 4-5 Queue Rear

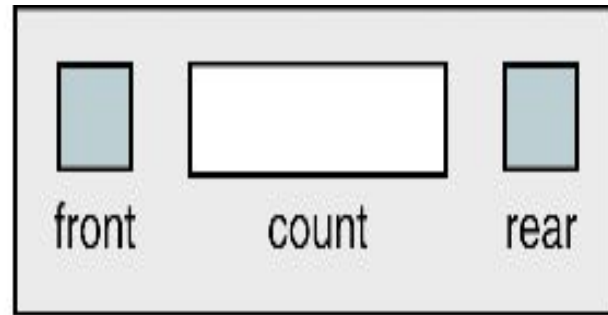


(a) Conceptual queue

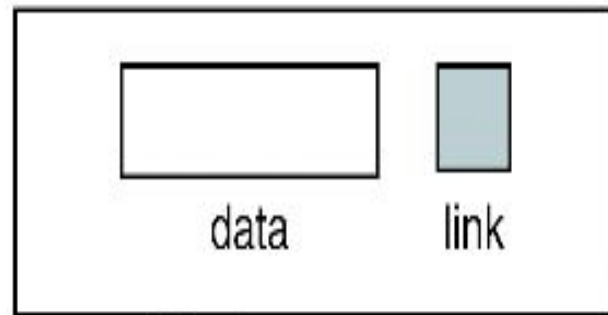


(b) Physical queue

FIGURE 4-7 Conceptual and Physical Queue Implementations



Head structure



Node structure

```
queueHead
  front
  count
  rear
end queueHead

node
  data
  link
end node
```

FIGURE 4-8 Queue Data Structure

FIGURE 4-9 Basic Queue Functions

Create queue



Enqueue



Enqueue

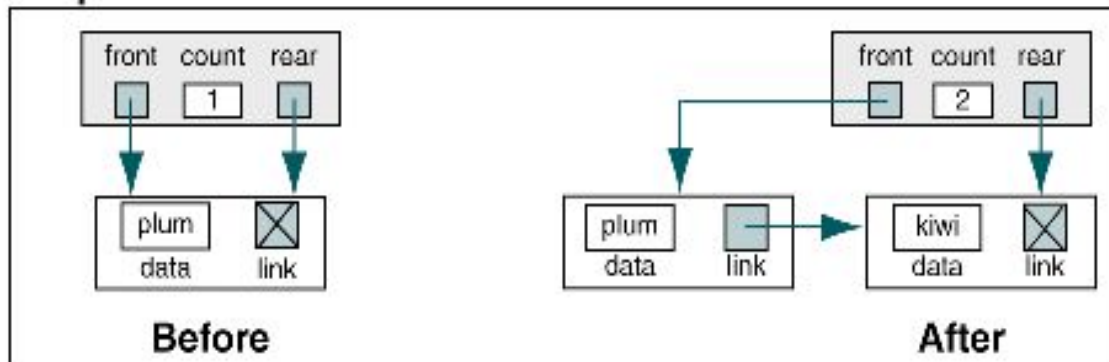
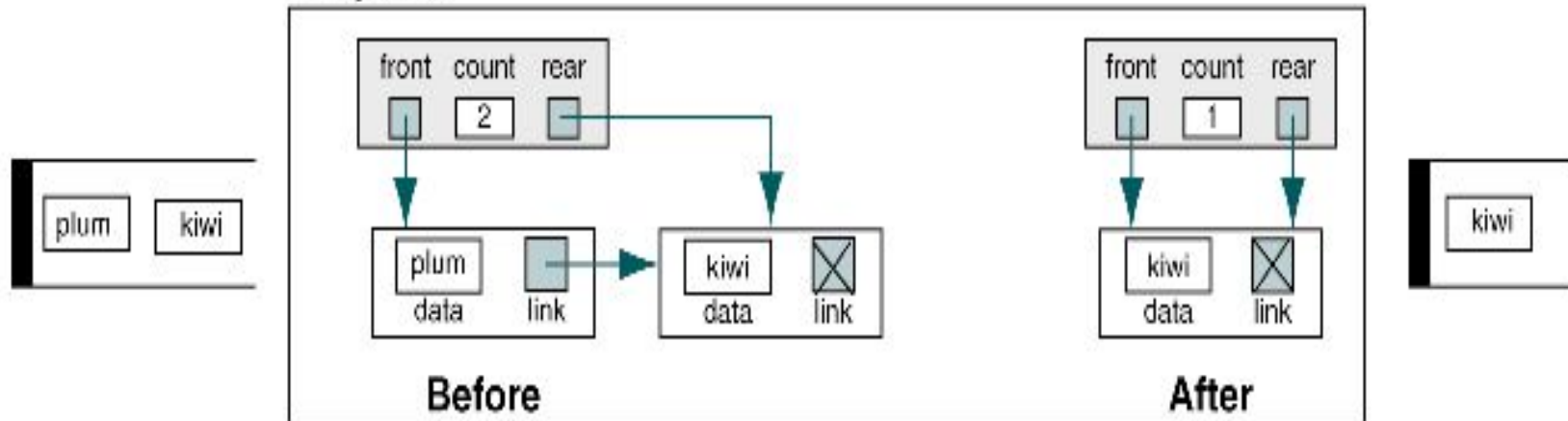
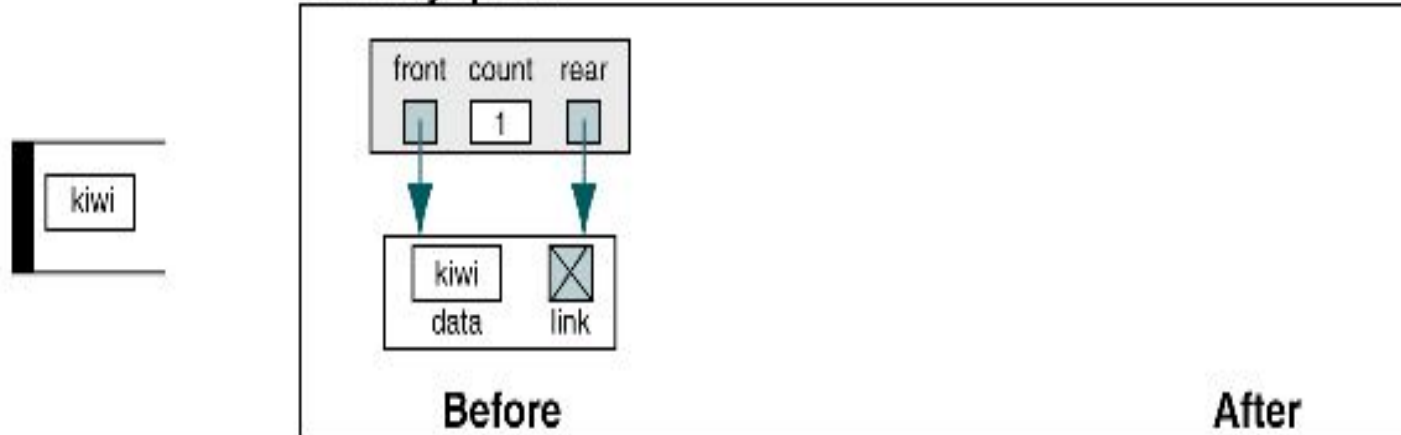


FIGURE 4-9 Basic Queue Functions (Continued)

Dequeue



Destroy queue



PROGRAM 4-1 Queue ADT Data Structures

```
1  //Queue ADT Type Defintions
2      typedef struct node
3      {
4          void*          dataPtr;
5          struct node* next;
6      } QUEUE_NODE;
7      typedef struct
8      {
9          QUEUE_NODE* front;
10         QUEUE_NODE* rear;
11         int          count;
12     } QUEUE;
13
14 //Prototype Declarations
15 QUEUE* createQueue (void);
16 QUEUE* destroyQueue (QUEUE* queue);
17
18 bool dequeue (QUEUE* queue, void** itemPtr);
19 bool enqueue (QUEUE* queue, void* itemPtr);
20 bool queueFront (QUEUE* queue, void** itemPtr);
21 bool queueRear (QUEUE* queue, void** itemPtr);
22 int queueCount (QUEUE* queue);
23
24 bool emptyQueue (QUEUE* queue);
25 bool fullQueue (QUEUE* queue);
26 //End of Queue ADT Definitions
```

PROGRAM 4-2 Create Queue

```
1  /*===== createQueue =====
2   Allocates memory for a queue head node from dynamic
3   memory and returns its address to the caller.
4   Pre    nothing
5   Post   head has been allocated and initialized
6   Return head if successful; null if overflow
7  */
8  QUEUE* createQueue (void)
9  {
10 //Local Definitions
11     QUEUE* queue;
12
13 //Statements
14     queue = (QUEUE*) malloc (sizeof (QUEUE));
15     if (queue)
16     {
17         queue->front  = NULL;
18         queue->rear   = NULL;
19         queue->count  = 0;
20     } // if
21     return queue;
22 } // createQueue
```

PROGRAM 4-3 Enqueue

```
1  /*===== enqueue =====  
2    This algorithm inserts data into a queue.  
3    Pre    queue has been created  
4    Post   data have been inserted  
5    Return true if successful, false if overflow  
6  */
```

continued

PROGRAM 4-3 Enqueue (continued)

```
7  bool enqueue (QUEUE* queue, void* itemPtr)
8  {
9      //Local Definitions
10     QUEUE_NODE* newPtr;
11
12     //Statements
13     if (!(newPtr =
14         (QUEUE_NODE*)malloc(sizeof(QUEUE_NODE))))
15         return false;
16
17     newPtr->dataPtr = itemPtr;
18     newPtr->next    = NULL;
19
20     if (queue->count == 0)
21         // Inserting into null queue
22         queue->front = newPtr;
23     else
24         queue->rear->next = newPtr;
25
26     (queue->count)++;
27     queue->rear = newPtr;
28     return true;
29 } // enqueue
```

```

8  bool dequeue (QUEUE* queue, void** itemPtr)
9  {
10 //Local Definitions
11     QUEUE_NODE* deleteLoc;
12
13 //Statements
14     if (!queue->count)
15         return false;
16
17     *itemPtr = queue->front->dataPtr;
18     deleteLoc = queue->front;
19     if (queue->count == 1)
20         // Deleting only item in queue
21         queue->rear = queue->front = NULL;
22     else
23         queue->front = queue->front->next;
24     (queue->count)--;
25     free (deleteLoc);
26
27     return true;
28 } // dequeue

```

PROGRAM 4-5 Queue Front

```
1  /*----- queueFront -----
2   This algorithm retrieves data at front of the
3   queue without changing the queue contents.
4   Pre    queue is pointer to an initialized queue
5   Post   itemPtr passed back to caller
6   Return true if successful; false if underflow
7  */
8  bool queueFront (QUEUE* queue, void** itemPtr)
9  {
10     //Statements
11     if (!queue->count)
12         return false;
13     else
14     {
15         *itemPtr = queue->front->dataPtr;
16         return true;
17     } // else
18 } // queueFront
```

PROGRAM 4-6 Queue Rear

```
1  /*----- queueRear -----  
2  Retrieves data at the rear of the queue  
3  without changing the queue contents.  
4      Pre    queue is pointer to initialized queue  
5      Post   Data passed back to caller  
6      Return true if successful; false if underflow  
7  */  
8  bool queueRear (QUEUE* queue, void** itemPtr)  
9  {  
10     //Statements  
11     if (!queue->count)  
12         return true;
```

continued

PROGRAM 4-6 Queue Rear (*continued*)

```
13     else  
14     {  
15         *itemPtr = queue->rear->dataPtr;  
16         return false;  
17     } // else  
18 } // queueRear
```

PROGRAM 4-8 Full Queue

```
1  /*----- fullQueue -----  
2  This algorithm checks to see if queue is full. It  
3  is full if memory cannot be allocated for next node.  
4  Pre   queue is a pointer to a queue head node  
5  Return true if full; false if room for a node  
6  */  
7  bool fullQueue (QUEUE* queue)  
8  {
```

PROGRAM 4-8 Full Queue (continued)

```
9  //Local Definitions  
10 QUEUE_NODE* temp;  
11  
12 //Statements  
13 temp = (QUEUE_NODE*)malloc(sizeof(*(queue->rear)));  
14 if (temp)  
15 {  
16     free (temp);  
17     return true;  
18 } // if  
19 // Heap full  
20 return false;  
21 } // fullQueue
```

PROGRAM 4-9 Queue Count

```
1  /*----- queueCount -----
2   Returns the number of elements in the queue.
3   Pre   queue is pointer to the queue head node
4   Return queue count
5  */
6  int queueCount(Queue* queue)
7  {
8   //Statements
9   return queue->count;
10 } // queueCount
```

PROGRAM 4-10 Destroy Queue (continued)

```
5   Post   All data have been deleted and recycled
6   Return null pointer
7  */
8  Queue* destroyQueue (Queue* queue)
9  {
10 //Local Definitions
11   Queue_Node* deletePtr;
12
13 //Statements
14   if (queue)
15   {
16     while (queue->front != NULL)
17     {
18       free (queue->front->dataPtr);
19       deletePtr = queue->front;
20       queue->front = queue->front->next;
21       free (deletePtr);
22     } // while
23     free (queue);
24   } // if
25   return NULL;
26 } // destroyQueue
```

Drawback of Linear Queue

- Once the queue is full, even though few elements from the front are deleted and some occupied space is relieved, it is not possible to add anymore new elements, as the rear has already reached the Queue's rear most position.

Circular Queue

- This queue is not linear but circular.
- Its structure can be like the following figure:

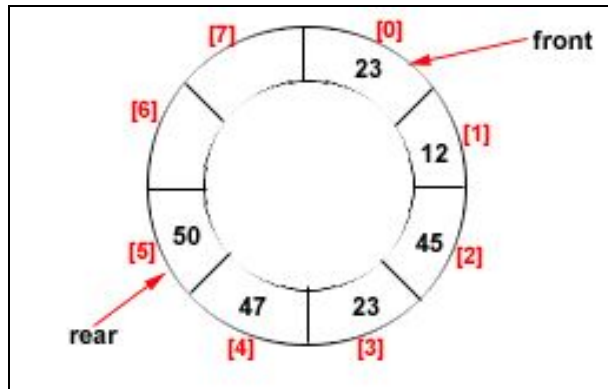


Figure: Circular Queue having Rear = 5 and Front = 0

Insert-Circular-Q(CQueue, Rear, Front, N, Item)

1. If $\text{Front} = 0$ and $\text{Rear} = 0$ then Set $\text{Front} := 1$ and go to step 4.
2. If $\text{Front} = 1$ and $\text{Rear} = N$ or $\text{Front} = \text{Rear} + 1$
then Print: "Circular Queue Overflow" and Return.
3. If $\text{Rear} = N$ then Set $\text{Rear} := 1$ and go to step 5.
4. Set $\text{Rear} := \text{Rear} + 1$
5. Set $\text{CQueue}[\text{Rear}] := \text{Item}$.
6. Return

Delete-Circular-Q(CQueue, Front, Rear, Item)

1. If Front = 0 then

Print: "Circular Queue Underflow" and Return. /*..Delete without Insertion

2. Set Item := CQueue [Front]

3. If Front = N then Set Front = 1 and Return.

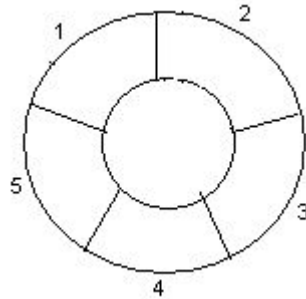
4. If Front = Rear then Set Front = 0 and Rear = 0 and Return.

5. Set Front := Front + 1

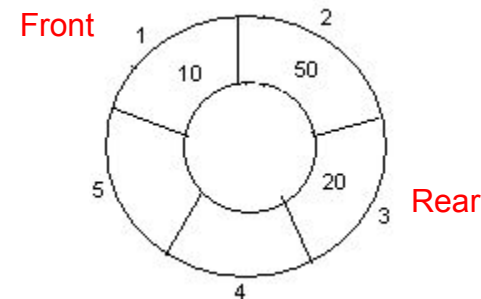
6. Return.

Example: Consider the following circular queue with $N = 5$.

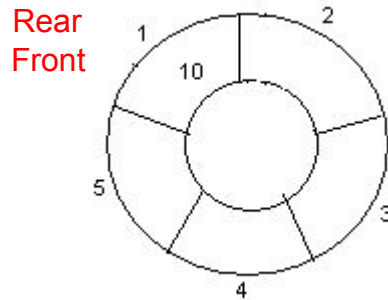
1. Initially, $\text{Rear} = 0$, $\text{Front} = 0$.



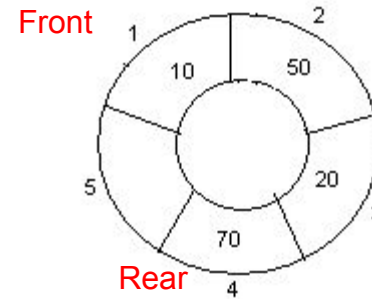
4. Insert 20, $\text{Rear} = 3$, $\text{Front} = 0$.



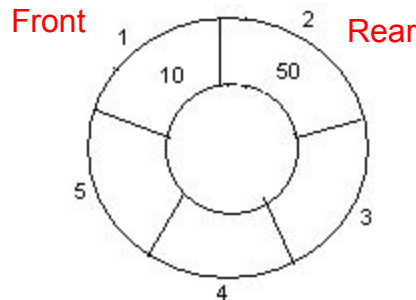
2. Insert 10, $\text{Rear} = 1$, $\text{Front} = 1$.



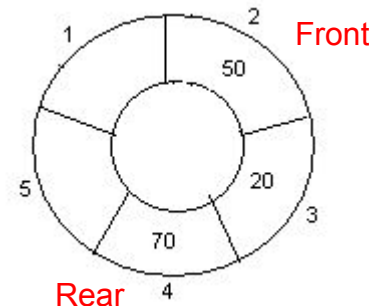
5. Insert 70, $\text{Rear} = 4$, $\text{Front} = 1$.



3. Insert 50, $\text{Rear} = 2$, $\text{Front} = 1$.



6. Delete front, $\text{Rear} = 4$, $\text{Front} = 2$.

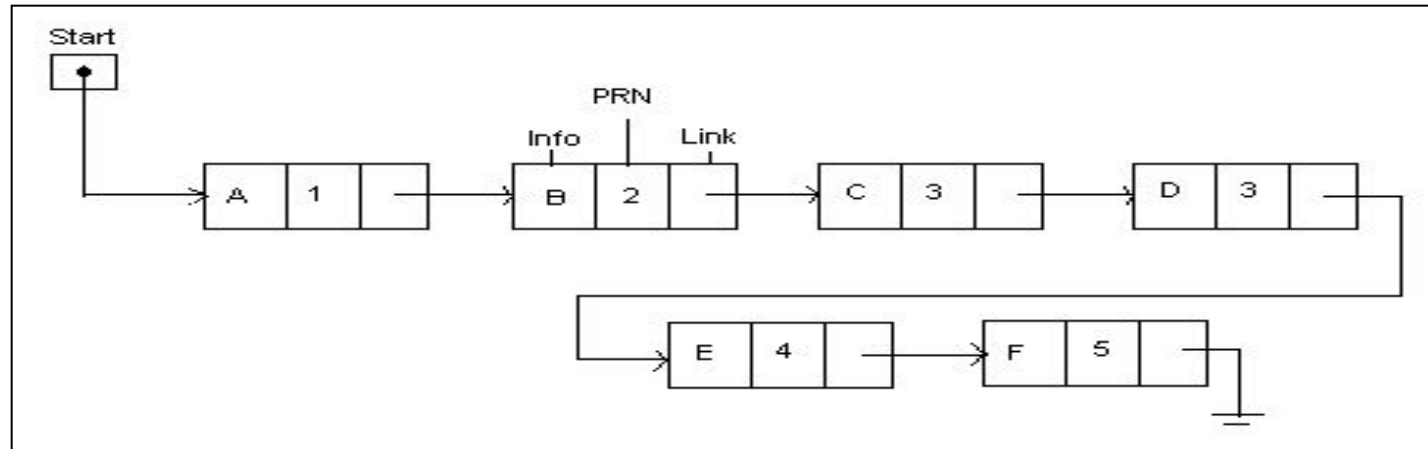


Priority Queue

- A priority queue is a collection of elements such that each element has been assigned a priority.
- A priority queue supports inserting new priorities, and removing the highest priority.
- Two elements with the same priority are processed according to the order in which they were added to the queue.

Representation of Priority Queue

- Each node in the list has three fields: an information field INFO, a priority number PRN and a link number Link.
- A node X precedes a node Y in the list when X has higher priority than Y or both have the same priority but X was added to the list before Y.



Priority Queue Implementation

Time Complexity Comparison

Implementation	Insert	Extract-Min/Max	Find-Min/Max
Unsorted Array	$O(1)$	$O(n)$	$O(n)$
Sorted Array	$O(n)$	$O(1)$	$O(1)$
Unsorted Linked List	$O(1)$	$O(n)$	$O(n)$
Sorted Linked List	$O(n)$	$O(1)$	$O(1)$

Deque

- A deque is a linear list in which data elements can be added or removed at either end but not in the middle.
- This type of queue is also known as dequeue and double-ended queue.
- There are two types of DEQUE.

Input-Restricted Deque – Allow insertions at only one end of the list but deletions at both ends of the list.

Output-Restricted Deque - Allow insertions at both ends but deletions at only one end of the list.

•**Example:**

0	1	2	3	4	5	6	7
60	20					100	80

Figure: Example of a Deque

Review Questions

- ❑ Suppose a circular queue is implemented using an array of size $N = 10$. Determine the number of elements in the queue for the following cases:
 - $FRONT = 3, REAR = 7$
 - $FRONT = 8, REAR = 2$
 - $FRONT = 5, REAR = 6$, then three elements are deleted.
 - If the queue is full and $FRONT = 4$, what will be the value of $REAR$?
- ❑ A priority queue is maintained as a sorted linked list. The queue initially contains the following elements: (lower priority values indicate higher priority)

Element	Priority
Alpha	2
Beta	5
Gamma	3
Delta	4

- Insert (Epsilon, 1) and (Zeta, 6) into the queue.
- Describe the new structure.
- Delete the highest-priority element and describe the updated queue.
- What if the priority queue was maintained as an unsorted linked list?

Review Questions

- ☐ A deque is implemented using an array of size $N = 6$. The current deque is represented as:
FRONT = 1, REAR = 4
____, A, B, C, D, ____
 - Insert E at the rear and describe the new structure.
 - Insert Z at the front and describe the updated deque.
 - Delete two elements from the front.
 - What is the new FRONT index?
- ☐ Compare **circular queues** with **regular queues** in terms of memory efficiency.
- ☐ Explain the advantages of using a **deque** instead of a **regular** queue.
- ☐ Write the logic part of an algorithm for -
 - Checking **overflow** and **underflow** in a circular queue.
 - Checking the **highest/lowest priority** item in a priority queue.
 - Checking the **size** of a queue.
- ☐ Give a real life example where you would prefer using a regular queue over priority queue or, a deque over a regular queue.